Building Serene from scratch

Repository: Serene-agent-from-scratch / create_agent | Stack: Qwen3-8B on Ollama, LangChain create_agent, nomic-embed-text

Serene is the coaching agent of the SoYi app, an emotional wellbeing companion for people who reach for food, cigarettes, or alcohol when something deeper is unmet. Due to heavy regulation on health apps, it currently uses hard-coded text.

This project is a minimal example from my testbed for building a real agent to eventually replace it.

What this document is. A high-level account and explanation of what I built in each step.

Contents

- 1. The big picture (layers, repository layout, vocabulary, where I am)
- 2. Step 1: a single call through a LangChain chat model
- 3. Step 2: the smallest create_agent
- 4. Step 3: a voice, via the system prompt
- 5. Step 4: tools and the tool-calling round trip
- 6. Step 5: memory with a checkpointer and thread_id
- 7. Step 6: knowledge retrieval and agentic RAG
- 8. The approach as a whole, known gaps, practice questions

1. The big picture

I built a coaching agent, Serene, in six steps, each layering on exactly one capability, to see what it contributed.

I ran an open-weight LLM model, Qwen3-8B, on a local program, Ollama. Everything else is scaffold and harness around it. In particular:

ChatOllama and OllamaEmbeddings are LangChain wrappers that turn my messages into HTTP requests, send them to Ollama, and wrap the replies as AIMessage objects or vectors.

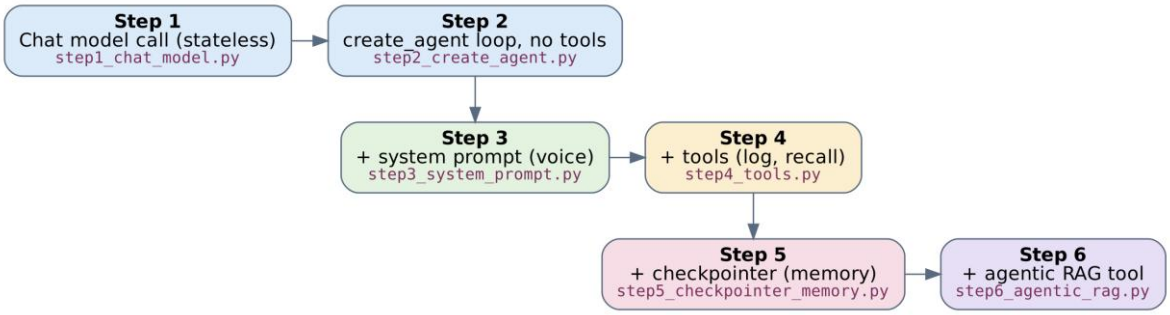


Figure 1. The six layers, each with the script that implements it

(As shown in Step 1) The LLM is a stateless text function which remembers nothing. Everything that looks like intelligence in the product is built in the code around it and reaches the model only as messages. The persona is a system message (Step 3). Tool use is a request from the model, run by the framework and returned as a tool message (Step 4). Memory is the saved history replayed into each call (Step 5). Knowledge is retrieved text returned as a tool message (Step 6).

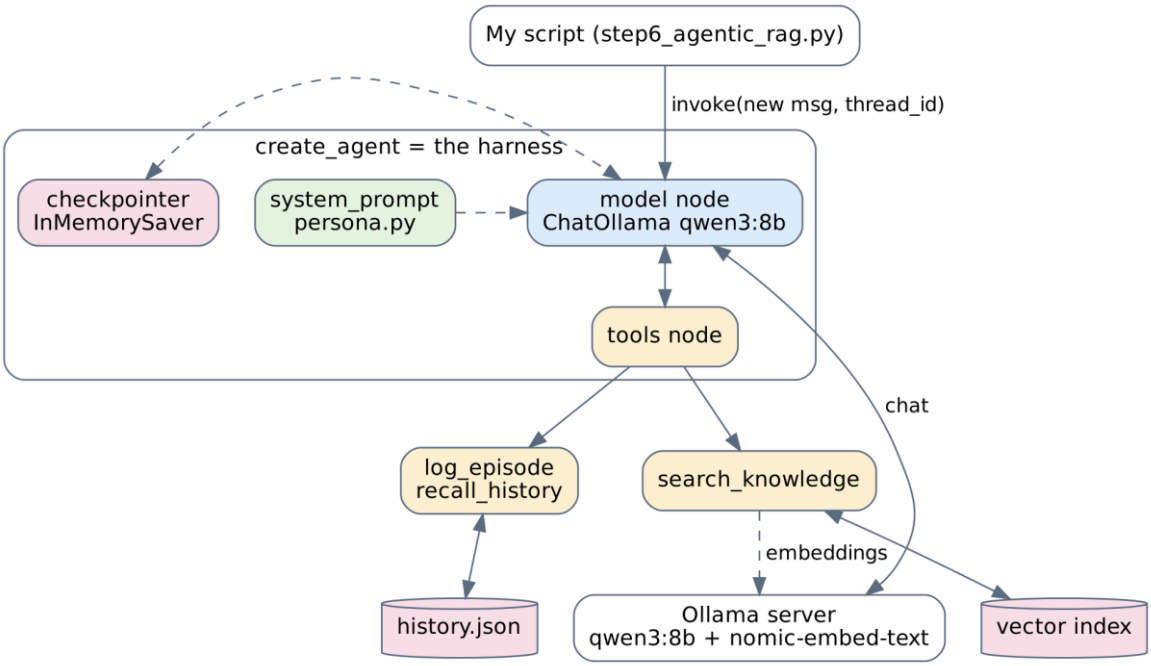


Figure 2. The agent as it stands after Step 6

Repository layout

Path	What it holds
create_agent/	The step-by-step build with LangChain create_agent: step1 to step6 scripts, shared messages.py, eval_retrieval.py, and the serene/ package (persona.py, tools.py, knowledge.py).
langgraph/	Reserved for the same agent rebuilt on LangGraph. Empty for now.
knowledge/	The knowledge base (Serene_Knowledge.md) shared by both builds.
output/	Saved run outputs per step.
README.md, .gitignore	Project overview and run instructions. The .gitignore excludes secrets, caches, run outputs, history.json, my guidance notes and *.docx.

Vocabulary

Term	Meaning	In my project
LLM	The trained model. Stateless: text in, text out.	Qwen3-8B
Model server	Program that loads the LLM and answers requests.	Ollama
Chat model (LangChain)	Wrapper class giving a common interface: messages in, AIMessage out.	ChatOllama
Agent	The LLM autonomously using tools in a loop to reach the defined goal	what create_agent builds
Harness	The code around the LLM that runs the loop, executes tools and keeps state.	create_agent plus (later) my middleware
Scaffold	What I supply to steer the LLM: system prompt, tool docstrings.	persona.py, tool docstrings

2. Step 1: a single call through a LangChain chat model

What I implemented

`step1_chat_model.py` creates `ChatOllama(model="qwen3:8b", temperature=0.2, ...)` twice, once with `reasoning=False` and once with `reasoning=True`. Each is invoked with a simple system message (`COACH_PROMPT`) followed by the three shared user turns from `messages.py`.

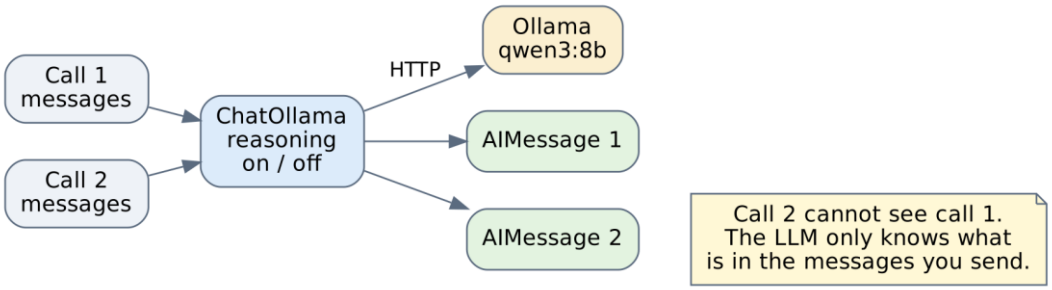


Figure 3. Each invoke is independent; the LLM sees only what I send

What the test outputs show

- The outputs are generic and over-helpful: sympathy, offers to "explore gentle ways", emoji, and in the reasoning run a list of suggestions.

Pros

- Smallest possible surface: exactly one request and one reply.
- The interface is provider-neutral, so swapping the model class later is cheap.
- Local model: no per-token cost, and data stays on my machine.

Cons / limits

- No memory, no tools, no persona: it cannot behave like an assistant yet.
- I only saved `.content`; the metadata (`usage_metadata`, `response_metadata`) that I could inspect was not saved.

Learning notes

- Stateless is the foundation. Two invokes are two unrelated conversations. Step 5 exists because of this.

- Message roles are system, human (user), ai, plus tool. Strings, tuples and dicts are all coerced into message objects.

- Reasoning models spend tokens thinking. Any output cap counts those tokens too, which caused trouble in later stages.

- **What value the LangChain wrapper adds over calling the API directly:** A common message and model interface, tool and agent integration, tracing hooks, and the ability to swap providers. The cost is a layer that hides the wire format.

- **"Does the model remember?"** No.

3. Step 2: the smallest create_agent

What I implemented

`step2_create_agent.py` passes a `ChatOllama` to `create_agent()` and calls `agent.invoke({"messages": messages})`. I print the graph with `get_graph().draw_ascii()` and write every message with its class name (`type(msg).__name__`) to `output/step2_output.md`.

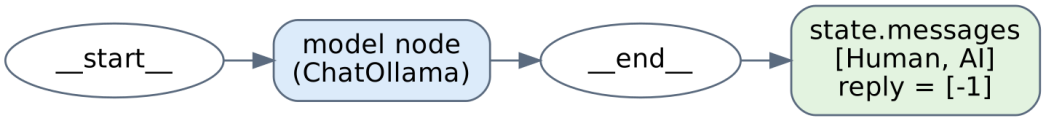


Figure 4. With no tools, the agent loop runs the model once

What the test outputs show

- The ASCII graph is `__start__`, `model`, `__end__`: a compiled LangGraph graph with a **single node**.
- The result is a dict; the reply is an `AIMessage accessible at: result["messages"][-1]`.
- The reply to "I'm never going to look good again" is a bulleted list of advice. A one-line "calm coach" prompt does not constrain an 8B model much.

Pros

- Shows that an agent is a graph with state, not magic.
- Zero extra code to later add tools, memory and middleware: they are arguments to the same function.

Cons / limits

- With `no tool`, it is functionally the same as Step 1 plus a system prompt.
- The graph is a black box until I print it or trace it.

Learning notes

- `create_agent` returns a compiled graph. State is a dict whose `messages` list accumulates; that list is the whole conversation.
- `system_prompt=` is the right place for the system text; I should not also put a system message in the list or it is duplicated.

4. Step 3: a voice, via the system prompt

What I implemented

The agent still has no tools.

I properly defined `SERENE_PROMPT in serene/persona.py` and pass it as `system_prompt in step3_system_prompt.py`. The prompt has sections: role, tools (what each is for), hard rules, voice, boundaries (crisis and dependency) and expectation under uncertainty.

The prompt in the repo now differs from the one the saved Step 3 output was made with: I later removed the forbidden-word list as it doesn't work with such a small LLM model, Qwen3-8B, added nuances like "motherly compassion", and made the knowledge search mandatory when a craving, slip, feeling or self-judgement appears.

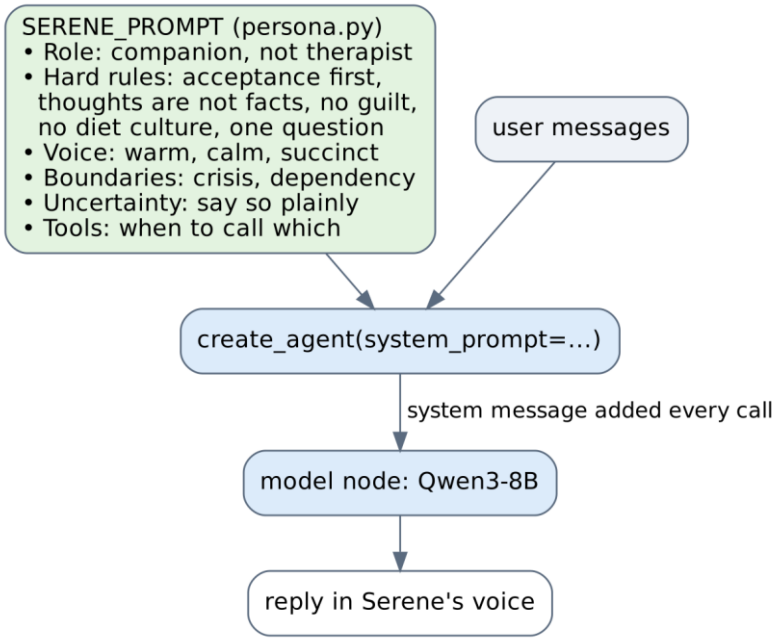


Figure 5. The prompt is the scaffold added in front of every model call

> **What the test outputs show**
>
> - The reply does say "the thought is not a fact", so the rule has some effect.
> - The reply contains a sample phrase from the prompt, copied word for word.
> - In later Step 4 (reasoning off) the reply "What do you think was going on there?" is also a verbatim prompt example.

Pros	Cons / limits
• Cheapest lever there is: no infrastructure, immediate change in tone. • Rules are readable and versionable in one file.	• Soft control: an 8B model follows long rule lists unevenly, and nothing enforces them. • Examples in the prompt become templates in the output. • The model knows nothing about the domain beyond the prompt (fixed by RAG in later Step 6).

Learning notes

- Add a rule because a failure made me need it, not in advance.
- Say "write fresh each time" if I include sample phrases, or leave samples out.
- Instructions are not guarantees. That is the motivation for the hard checks by guardrail middleware.

5. Step 4: tools and the tool-calling round trip

What I implemented

`serene/tools.py` defines two `@tool` functions and passes them to `create_agent`():

- `log_episode(behaviour, trigger, choice, note="")`, which appends a dict to `serene/history.json` and returns "Logged.", and
- `recall_history(limit=10)`, which recalls the last entries from the history of logged episodes.

This introduces a very basic and selective long-term memory mechanism.

Plus semi-manual incremental historical conversation management.

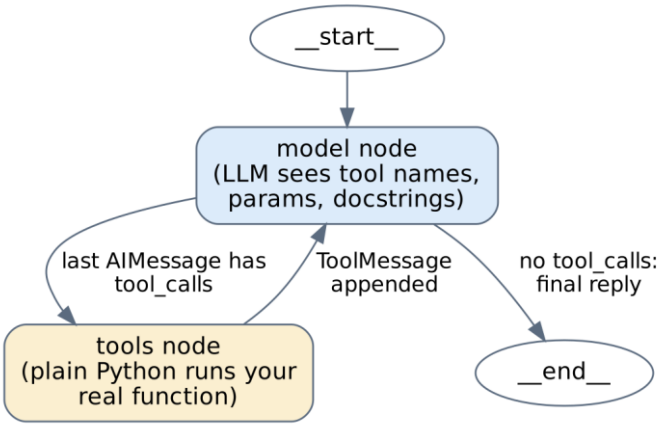


Figure 6. The loop now has a tools node and a conditional edge

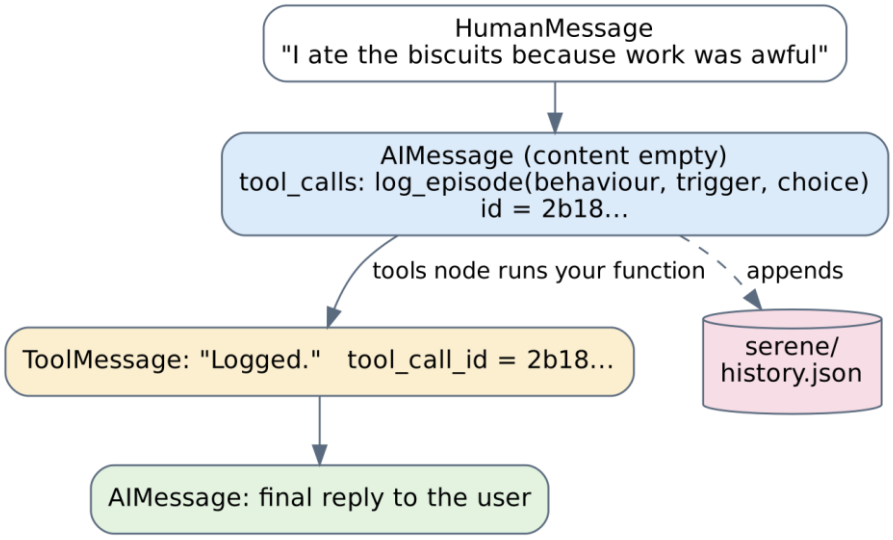


Figure 7. The message chain for one tool call

Who does what (the key mental model)

#	Step
1	The harness sends the tool names, parameter schemas and docstrings to the LLM on every request.
2	The LLM writes text asking for a tool: a name and arguments. It runs nothing itself.
3	ChatOllama parses that into `AIMessage.tool_calls` (name, args, id).
4	The tools node (ordinary Python) calls my real function with those arguments.
5	The return value becomes a `ToolMessage` with the matching id, appended to messages.
6	The model is called again with everything so far, and either replies or asks for another tool.

> **What the test outputs show**
>
> - **Reasoning on:** on "I ate the biscuits because work was awful" the model emitted `log_episode(behaviour="ate the biscuits", trigger="work was awful", choice="ate the biscuits")`, got "Logged.", then replied. This is the full Human, AI(tool_call), Tool, AI chain. The arguments are sensible, though `choice` just repeats `behaviour`.
> - **Reasoning off (separate file):** in that single run the model never called the tool, and one reply is empty. It is one sample each though, so not conclusive.
> - Reflective turns (e.g. "I'm never going to look good again") correctly triggered no logging.

Pros

- Clear, visible round trip; the saved message list is a good teaching artefact.
- Docstring-as-instruction is simple: the model needs no code access.
- Tools are plain Python, easy to unit test without the model.

Cons / limits

- No timestamp in entries, no file locking, no validation. Fine for a POC, not for production.
- Small models under-call or mis-fill tools; docstring precision matters a lot.
- Script-owned history grows without bound and is lost when the process ends.

Learning notes

- How function calling is done: With every request, the framework tells the model about the tools *. If the LLM decides a tool would help, it doesn't run it directly, but generates a structured request: a tool name plus JSON arguments. The chat model class parses that request into tool_calls on the AI message. The framework's tools node then validates the arguments against the schema, calls the tool function, and returns the result to the model as a tool message.

- *The docstring is the interface that tells the model what a tool does, and a parameter schema built from the type hints constrains what the arguments can be.

- An `AIMessage` that requests a tool usually has empty `content`. Printing only `.content` shows a blank, which is easy to mistake for a failure.

- Reasoning tokens count against the argument `num_predict`. A tight cap can truncate before the tool call is written.

- Experiments still worth doing: vague versus precise docstring, no docstring, "hello", and counting tool calls over several runs.

6. Step 5: memory with a checkpointer and thread_id

What I implemented

Added `checkpointer=InMemorySaver()` to `create_agent()`, using LangGraph's checkpointer mechanism to enable the agent's memory. The conversation state is saved after each graph step, keyed by thread_id. Different thread ids are separate conversations.

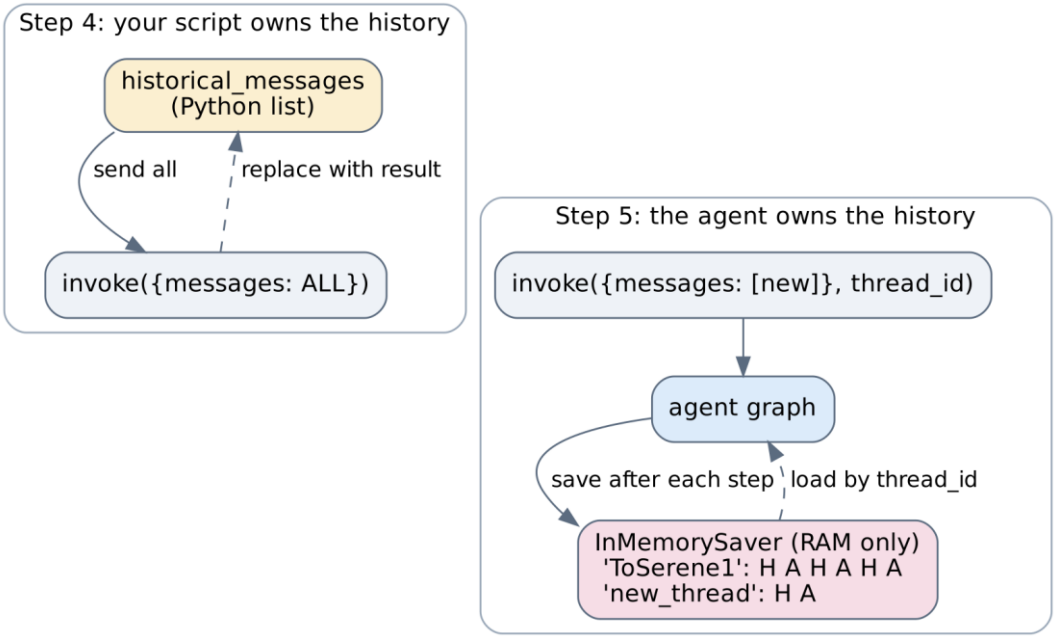


Figure 8. Same job, different owner: who stores the history

On each call I send only a message and a thread_id, and the framework loads that thread's saved history, adds the new message and runs the agent. The saver is in RAM only, so the history is lost when the program exits.

A separate test call with just the last message on `thread_id="new_thread"` checks that the agent "in" a different thread knows nothing about this thread indeed.

> **What the test outputs show**
>
> - My output file shows earlier messages repeated each turn, so memory works.
> - The new thread answered "I'm never going to look good again" with no reference to stress or biscuits, so isolation works.
> - In one run the biscuits turn returned an **empty** reply with no tool call. In another run the same turn called `log_episode`. I have not confirmed the cause; a reasoning-heavy model with an output cap is one suspect, and run-to-run variation is another.

Pros

- User-friendliness: The framework owns history & threads isolate conversations.
- Checkpoints are saved at every graph step, enabling resume, inspection, rewind and human-in-the-loop later.
- Swapping to SQLite or Postgres makes it survive restarts without changing the agent code.

Cons / limits

- `InMemorySaver` is RAM only: memory is lost when the process ends.
- The full history is still resent to the LLM on every call, so cost and latency grow with conversation length and eventually hit the context window. Memory management (trimming, summarising) is not solved here.
- This is thread memory only. It is not long-term memory across conversations (that is what `history.json` partly does).

Learning notes

- A **checkpointer** is the persistence layer of the agent's graph. While the agent runs, LangGraph keeps a state object. The checkpointer saves a snapshot of that state, called a checkpoint, after each step of the graph, keyed by a thread_id.

When I call invoke(thread_id), the framework loads the latest checkpoint for the thread in question, topping it up with the new query message, runs the graph, and saves the updated state again.

Checkpointer scope is a thread. A thread id is the primary key of a conversation. A different thread_id has its own empty state, which is why new_thread knew nothing about ToSerene1.

- **Memory: "How does a LLM remember?"** It does not; it's achieved by the harness. I.e., the agent's memory is the framework replaying the saved history into the model's input on every call.

- Step 4 and Step 5 have the same data flow. Both resend the whole history; only the owner of the action changes.

- Two kinds of memory now exist in my project: short-term (checkpointer) and a crude long-term (`history.json` via tools).

- **"What to improve for production?"** A persistent checkpointer (Postgres or SQLite), context trimming or summarisation, and a separate long-term store with retrieval.

7. Step 6: knowledge retrieval and agentic RAG

The five RAG stages are Load, Chunk, Embed, Store, Retrieve.

I built the retriever (7a) and exposed it as a tool the agent may call (7b), enabling its Retrieval Augmented Generation.

7a. The retriever (serene/knowledge.py)

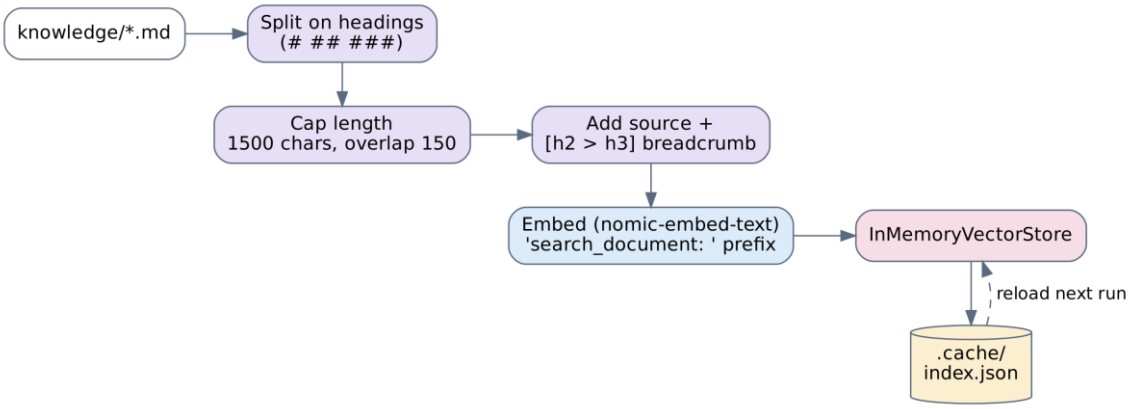


Figure 9. Indexing pipeline: load, chunk, embed, store

Stage	What my code does
Load	`Path.rglob("*.md")` over the shared `knowledge/` folder.
Chunk	`First split by headers, then split with defined max size.` Each chunk gets `source` metadata and a prefix such as `[Trigger: Pressure-Driven > ...]` built from headers h2 and h3 (document title h1 is skipped).
Embed	A `NomicEmbeddings` subclass of `OllamaEmbeddings` that prepends `search_document:` to chunks and `search_query:` to queries, because nomic-embed-text expects those task prefixes.
Store	`InMemoryVectorStore.from_documents`, dumped to `create_agent/serene/.cache/index.json` (about 1.6 MB, git-ignored) and reloaded on later runs. A module-level `_store` makes it a lazy singleton.
Retrieve	`search(query, k=4)` calls `similarity_search`.

I performed a quick `eval`, which runs queries of different nature and nuances through `similarity_search_with_score()` and prints rank, score, source and section.

That separates "did retrieval fail?" from "did the model fail?", which is the point of building the retriever alone first.

7b. Retrieval as a tool (serene/tools.py and step6_agentic_rag.py)

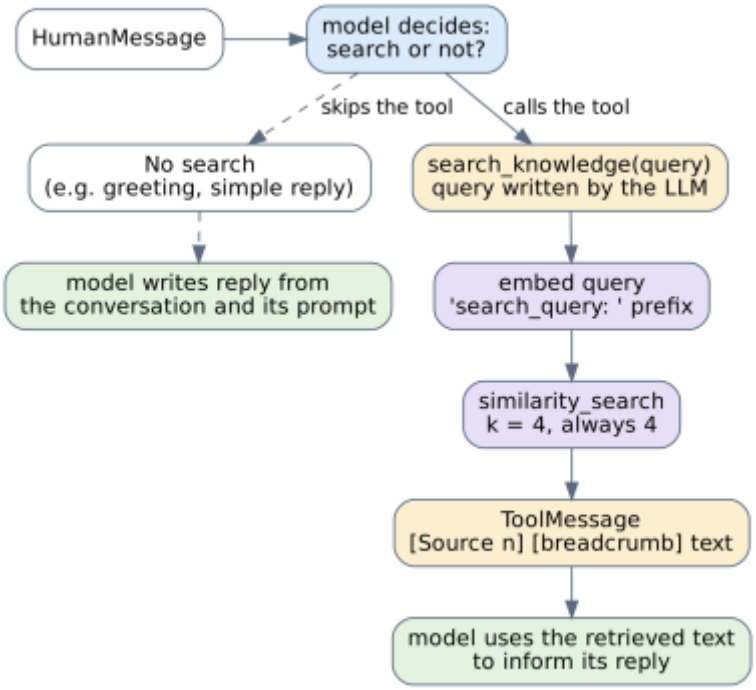


Figure 10. Agentic RAG: the model chooses to search and writes the query

The LLM decides to use the retriever or not, before it generates.

`search_knowledge(query)` returns the retrieved chunks as one whole string, each chunk labelled `[Source n: filename]`.

The agent now has 3 tools and the checkpointer.

> **What the test outputs show**
>
> - **Turn 1** ("I'm stressed. I can't stop eating."): no tool call, despite the prompt telling the model to search first. A generic but on-voice reply.
>
> - **Turn 2** (biscuits): the model searched for "eating when stressed from work" and retrieved more relevant chunks (Pressure-Driven) and less relevant ones (Procrastination, Fatigue) . The Fatigue chunk contains diet-style advice. It failed to call `log_episode`.
>
> - **Turn 3** ("never going to look good again"): query "self-judgment about appearance during stress". Results were highly relevant (Rule 2 thoughts are not facts; Rules 6 and 7 no diet culture) and the reply visibly applied them.
>
> - **Rule slippage:** the replies contain several questions, an emoji, and "Would you like to..." offers, against the "one question at most" voice rule. A guardrail or evaluation would catch them.

Pros

- Retrieval is testable independably without the model (eval_retrieval.py).
- Heading-aware chunking keeps one idea per chunk; breadcrumbs add context to each chunk.
- Query and document prefixes (search_query: / search_document:) follow nomic-embed-text's expected usage, which should give better retrieval than embedding raw text.
- The model decides when to search, it is flexible and cheap on greetings.
- Cached index and fully local stack: fast restarts, private data.

Cons / limits

- Fixed k, no relevance threshold or grader. Similarity search always returns k results, even for an off-topic question.
- Whether the model calls the tool is probabilistic. Making the prompt say MUST increases chances but not a guarantee.
- Retrieved text is untrusted input to the model. Anything in the knowledge base can influence behaviour, so it is also a prompt-injection surface.
- In-memory store with a JSON dump does not scale.
- No re-ranking, no hybrid keyword search.

Learning notes

- The embedding model turns a piece of text into a vector. It decides what "similar" means.

- Compatibility:
    - The retrieved chunks come back to the LLM as plain text in a ToolMessage. The LLM never sees the vectors, so it doesn't have to match the embedding model. But:
    - Query and Document Embeddings Must Match.
    - Dimensionality Must Match between the embedding model and Vector DB.

- Documents and queries are embedded differently for nomic-embed-text. Missing the prefixes quietly lowers quality.

- Inspect scores, not only ranks. `eval_retrieval.py` is a seed for a proper retrieval evaluation (a fixed query set with expected sources, then hit rate at k).

8. To be continued …

Known gaps to close at a later stage

- No automated evaluation yet. Results are from single runs.
- No guardrail: the behaviours the prompt forbids (multiple questions, emoji, advice) do occur, and nothing catches them.
- No tracing: I cannot yet answer "what query did she send and what came back" from a trace; I rely on printed messages.
- `history.json` still uses a relative path, so scripts must be run from `create_agent/`.
- The domain is emotional wellbeing. The prompt has a crisis boundary, but it is untested, and an LLM-only check is not a substitute for escalation and human review.

My next steps

- Move agent construction into `serene/agent.py` (`build_agent()`), and use `Path(__file__).parent` for `history.json`.
- Do Step 1b (raw HTTP and the OpenAI SDK) so I can compare 3 ways of making the same call.
- Re-run Steps 1 to 6 so saved outputs match the current code, and count tool calls over 3 runs each.
- Turn `eval_retrieval.py` into a small labelled test: query, expected source, pass or fail.